

1. Use reduction to prove that the language $L = \{p(M1)p(M2) : L(M1) \cap L(M2) = \emptyset\}$ is not decidable (M1 and M2 are Turing machines, of course).

- Emptiness Theorem: $E_{TM} = \{p(M) : L(M) = \emptyset\}$ and that E_{TM} is undecidable
- Need to reduce E_{TM} to L to show that L is undecidable by showing that if L can be decided, then E_{TM} can be decided.
- Given M , construct two Turing machines $M1$ and $M2$ where $M1 = M$ and $M2$ is a Turing machine that accepts all strings Σ^*
 - $L(M2) = \Sigma^*$ and $L(M1) \cap L(M2) = L(M1) \cap \Sigma^* = L(M1)$
 - Thus $L(M1) \cap L(M2) = \emptyset$ iff $L(M1) = \emptyset$
- Now the string is $w = p(M1)p(M2)$
- M_L is the decider for L . On input $p(M)$, build $w = p(M1)p(M2)$ and run $M_L(w)$
- If it accepts then accept. If it rejects then reject.
- $w \in L$ iff $L(M) \cap L(M2) = \emptyset$ iff $L(M) = \emptyset$
- Therefore M_L decided L and hence $M_{E_{TM}}$ decides E_{TM} , but this is a contradiction as E_{TM} is proven to be undecidable through the emptiness problem.
- Therefore L is undecidable by proof of reduction from E_{TM} .

2. Are the following languages Turing-decidable? Turing-acceptable but not Turing-decidable? Not even Turing-acceptable? For each answer, give an explanation of your reasoning (just as in class, M is a generic deterministic Turing machine, w a generic input string to it, and p is an encoding function).

- $L1 = \{p(M) : |L(M)| > 10\}$
- $L2 = \{p(M) : |L(M)| \leq 10\}$
- $L3 = \{p(M)p(w) : M \searrow w \text{ in 10 steps or fewer}\}$

2A | $L1 = \{p(M) : |L(M)| > 10\}$:

- Set of encoded strings of TMs whose languages have more than 10 strings.

Turing Decidable:

- Given $P(m)$, can the TM determine if $L(M)$ contains more than 10 strings.
- Need to enumerate all strings in Σ^* and check whether M will accept more than 10 of them.
- M may never halt on some of the strings, so it might be possible that the TM could wait forever without knowing if the language of the machine is large enough.
- Suppose we could decide $L1$, then we could decide E_{TM} (empty problem machine):
 - Given $P(M)$, construct a decider for $L1$
 - If $|L(M)| > 10$ then output Yes. If $|L(M)| \leq 10$, enumerate all possible 10 string combinations and check if $|L(M)| = 0$
 - This would encounter the halting problem.
- Therefore, **$L1$ is NOT TURING DECIDABLE**

Turing-Acceptable:

- TM that accepts every string in the language and may run forever on strings not in language
- On input $P(M)$, enumerate all strings in Σ^* and simulate M on all strings.
- When M accepts new string, keep count. If count > 10 , accept $P(M)$
- If $|L(M)| > 10$, the machine will accept. If $|L(M)| \leq 10$, the machine runs forever.
- This meets the requirements for the language being turing acceptable, **therefore the language is turing acceptable.**

Therefore the language is semi-decidable as it is turing-acceptable but not turing-decidable.

2B | $L_2 = \{p(M) : |L(M)| \leq 10\}$:

Turing machine encoded strings where language recognized by the machine has at most 10 strings.

Turing Decidable:

- Given $P(m)$, can the TM determine if $L(M)$ contains less than or equal to 10 strings.
- On input $p(M)$:
 - List all strings w over Σ^*
 - Simulate M on each w to see which string it accepts and stop and accept if M has accepted at most 10 strings.
 - Reject if M has accepted more than 10 strings.
- M can never be sure that there are more strings to accept. May be infinite amount of time.
- M needs to check all infinite strings, which cannot be done in a numeric amount of time.
- Relates to the finite problem/

Therefore, L_2 is not turing-decidable.

Turing Acceptable:

- Can T_m accept the encodings where $|L(M)| \leq 10$
- On input $p(M)$:
 - Enumerate all strings over Σ^* and simulate M on each string.
 - Keep track of which strings M accepts and if there is ever more than 10, then reject.
- There lays a problem though. If M accepts less than 10 strings and loops forever processing the rest, then only the amount accepted will be known and not if another string will be found later.
- If M accepts at most 10 strings, then it will never halt and accept as it waits for another string.

Therefore L_2 is not turing-acceptable as it cannot always halt and accept in finite time

Hence, L_2 is not turing-decidable nor turing-acceptable.

$2C| L3 = \{p(M)p(w) : M \searrow w \text{ in 10 steps or fewer}\}$:

Set of all encodings $P(M)P(w)$ s/t TM M halts on input w within 10 steps or less.

Turing-Decidable:

On input $p(M)p(w)$, does M on input w halt in 10 steps or less?

- TM M on input w for ten steps or less:
 - If M halts by accepting or rejecting within 10 steps, accept it.
 - If M has not halted after 10 steps, reject it.
- 10 steps can be performed in simulation for a finite amount of time for any TM M and input w .
- Yes it does halt: After simulation 10 steps, M halts and answers yes or no based on whether it accepted or rejected the string yet. Always in finite amount of time.
- No undecidable or infinite loop issues exist for this M

Therefore the TM M does halt on input w in 10 steps or less and avoids problems like infinite loops and halting, so the language $L3$ is indeed decidable.

Turing-Acceptable:

Every decidable language is also acceptable. So yes, the language $L3$ is also acceptable.

Hence, the language $L3$ is both decidable and therefore acceptable.

3: Define an unrestricted grammar for the language $\{ww : w \in \{0, 1\}^*\}$. Explain clearly how it works or, even better, formally prove that it works.

Language consists of strings that has two halves that are identical

$\epsilon, 101101, 1111, 001001 \in L = \{ww : w \in \{0, 1\}^*\}$

- Need to generate a string of 0's and 1's and then copy it to create the second half.
- Generate string w on left and mark the spot where the copying will begin, then run through the steps of copying and move it to the right of the x to create the second half of the string w

Create a grammar $G=(V, T, S, P)$ where:

Terminals $T = \{0, 1\}$

Variables $X = \{S, X\}$ where S = start symbol and X = divider between first w and the second copied w .

Start Symbol = S

Production Rules $P =$

1. $S \rightarrow X$
2. $X \rightarrow 0x0$
3. $X \rightarrow 1x1$
4. $X \rightarrow \epsilon$

$G = (\{S, X\}, \{0, 1\}, S \{S \rightarrow X, X \rightarrow 0X0, X \rightarrow 1X1, X \rightarrow \epsilon\})$

- Start symbol S will rewrite to X
- At each step, choose to add two 0's or 1's—one to the left of the X and another to the right of the X

This unrestricted grammar will generate ww as it ensures that the same characters are generated to the left and right of the 'X' and the last rule makes sure it takes the 'X' out as it exits sentential form and becomes a string.

$\subseteq \wedge \in \forall \exists \neg \Sigma \varepsilon \cap \emptyset$

4II: Prove that the Turing-acceptable languages are closed under concatenation.

- Language L is turing-acceptable if TM exists that accepts every string in L , but can also loop forever on strings not in L
- Languages L_1 and L_2 . Concatenation: $L_1 * L_2 = \{xy : x \in L_1, y \in L_2\}$
- I need to prove that if L_1 and L_2 are turing-acceptable, then so is $L_1 * L_2$

- Assume L_1 and L_2 are turing-acceptable.
- Let M_1 be a TM that accepts L_1 and let M_2 be a TM that accepts L_2
- Need to now construct turing machine M that accepts $L_1 * L_2$

- Given input string w , split w into x and y where $x \in L_1$ and $y \in L_2$ and accept if for any of the splits, M_1 accepts x and M_2 accepts y .
- Now need to simulate the turing machine M .
- On input w :
- Loop from 0 to $|w|$ and set looping variable to k :
- Split w into $x = \{w[1 \dots k]\}$ and $y = w[k+1 \dots n]$ where $n = |w|$ and k is the splitting position
- For each of the splits. Simulate M_1 on x and M_2 on y .
- If both machines accept on a split, accept w .

- Then this is a turing machine that accepts $L_1 * L_2$
- If $w \in L_1 * L_2$, then $w = xy$ and $x \in L_1, y \in L_2$. For a split, M_1 and M_2 will accept, and then M consequently will also accept.
- If $w \notin L_1 * L_2$, then for every split, at least one of M_1 or M_2 does not accept. So M will never accept.
- This meets the requirements of being turing acceptable, therefore it is proven.

Therefore turing-acceptable languages have been proven to be closed under concatenation. If L_1 and L_2 are turing acceptable, then so is L_1L_2 .

State S_0 :